Python

22BCON1068

Assignment - 03

DATE: / /
PAGE NO.: 25

- Sec-A (2*5=10)

1. What is the main difference between Lists and tuples?

Ans The main difference between list and tuples is that lists are mutable and tuples are immutable. Few more differences are given.

LIST	Tuple
(i) List are mutable	(i) Tuple are immutable
(ii) List consume more memory.	Tuple consume less memory as compared to the list.
(iii) Lists have several built-in methods.	(ii) Tuple does not have many built-in methods.

2. Define a void function in Python.

Ans. In Python, a void function is a function that does not return any value. It performs a task or action without producing any output or result. Void functions are also known as procedures.

Code:

```
def voidOperation():
    num1 = int(input("Enter First Number:"))
    num2 = int(input("Enter second Number:"))
    sum = num1 + num2
    print("sum of %d and %d = %d" %(num1, num2, sum))
def main_fn():
    voidOperation()
    main_fn()
```

Output:

```
Enter First Number: 10
Enter second Number: 50
sum of 10 and 50 = 60
```

Apr 9, 2024, 22:35

22BC

2. What is the syntax for defining a function in python?

Ans In python, The syntax for defining a function using the "def" keyword followed by the function name, parameters (if any), and a colon (:) to start the function body.

Syntax:
```
def function_name (parameter 1, parameter 2, --):
    """Doc string explaining the function's purpose"""
```

4. What is a recursive function?

Ans A recursive function is a function that calls itself during its execution. This might sound like an infinite loop, but it's a powerful technique for solving problems that can be broken down into smaller, similar subproblems.

→ Recursive fun. typically consist of three parts:

(i) Base case

(ii) Recursive case

(iii) Update

5. What is a lambda function in python?

Ans In python, a lambda function is a small anonymous function that can have any number of parameters, but it can only have one expression.

it defined using the 'lambda' keyword.

Code:
```
add = lambda x, y : x+y

result = add(5, 3)
print (result)
```

Output: 8

Apr 9, 2024, 22:36

22BCON1068

DATE: / /
PAGE NO.: 27

- sec-B (7*3 = 21)

1. Describe three basic operations that can be performed on lists in Python.

Ans List are versatile data structure that allows for various operations. Here three basic operations that can be performed on lists:

(i) Attending Elements: Adding elements to the end of a list is a fundamental operation. This is typically done using the 'append()' method.

ex

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list)          o/p: [1, 2, 3, 4]
```

(ii) Accessing Elements: Accessing elements within a list is crucial. Elements in a list are indexed starting from 0. You can access elements using square brackets '[]'.

ex

```
my_list = [1, 2, 3, 4]
print(my_list[0])       o/p: 1
print(my_list[2])       o/p: 3
```

(iii) Modifying Elements: Lists in Python, are mutable ~~You~~ meaning we can change the value of elements after the lists is created. We can directly assign new values to specific indices in the list.

ex.

Apr 9, 2024, 22:36

```
my_list = [1, 2, 3, 4]
my_list[1] = 5
print(my_list)          o/p : [1, 5, 3, 4]
```

Q2. Define a recursive function in python and provide an example of its usage.

Ans A recursive function is a function that calls itself one or more time in its body. It's a powerful technique for solving problem that can be broken down into smaller, self-similar subproblems.

There are two case in recursive fun.

(i) Base Case: This is the simplest case that can be solved directly, without further recursion.

(ii) Recursive case: This is where the function calls itself, usually with a smaller or simpler version of the problem.

Example of Recursive fun.

```
①  def factorial(n):
②      if n == 0:
③          return 1
④      else:
⑤          return n * factorial(n-1)

print(factorial(5))
```

output: 120

in above example, 'factorial()' is a recursive fun. it calculates the factorial of a non-negative integer 'n', The base case is when 'n' equals 0, in which case the function return 1. otherwise, it returns 'n' multiplied by the factorial of 'n-1', thus calling itself recursively until it reaches the base case.

Q3. Explain how indexing and slicing work in lists and tuples.

Ans Indexing and slicing are fundamental operations for accessing and manipulating elements in lists and tuples in python.

- INdexing :

- Lists: Lists are ordered collections of items, and each item has an index associated with it. Indexing in list is zero-based, meaning the index of the first element is 0, the index of the second element is 1 and so on.

ex. my_list = [10, 20, 30, 40, 50]
print(my_list[0]) o/p : 10
print(my_list[2]) o/p : 30

- Tuples : Tuples are similar to list but immutable, meaning their elements cannot be changed after creation. Like lists, tuples support indexing and it follows the same zero-based indexing system.

Apr 9, 2024, 22:36

eg

```
my_tuple = (10, 20, 30, 40, 50)
print(my_tuple[0])        o/p : 10
print(my_tuple[2])        o/p : 30
```

- Slicing :
- Lists : Slicing allows to extract a subset of elements from a list. We use a colon (:) notation within square bracket [] to specify the starting and ending indices of the slice.

Syntax : [start : stop : step].

eg.

```
my_list = [10, 20, 30, 40, 50]
print(my_list[1:4])   o/p : [20, 30, 40]
print(my_list[:3])    o/p : [10, 20, 30]
print(my_list[2:])    o/p : [30, 40, 50]
print(my_list[:])     o/p : [10, 20, 30, 40, 50]
```

- Tuples : With tuples, slicing works as lists. it allows to extract a subset of elements from a tuples.

eg.

```
my_tuple = (10, 20, 30, 40, 50)
print(my_tuple[1:4])      o/p : (20, 30, 40)
print(my_tuple[:3])       o/p : (10, 20, 30)
print(my_tuple[2:])       o/p : (30, 40, 50)
print(my_tuple[:])        o/p : (10, 20, 30, 40, 50)
```

• Sec- C (3X11=33)

Q1 Discuss the advantages of using a tuple over a list in certain situations, with examples.

Ans Tuples offer several advantages over lists in specific scenarios where you want to prioritize data integrity, performance, or memory usage. Here we explain this -

(i) Immutable Nature: Tuples are immutable, meaning once they are created, their elements cannot be changed or modified. This immutability ensures data integrity, especially in situations where the data should not be altered accidentally.

ex. point = (10,20)
point[1] = 30 # This will raise a TypeError.

(ii) Performance: Due to their immutability, tuples are generally faster and more memory-efficient compared to lists.
tuples have a smaller memory footprint because they don't require extra space to store methods like lists do.

(iii) Safety in Data Integrity: Since tuples are immutable, they provide safety in scenarios where data should remain unchanged throughout the program execution. This can prevent accidental modification of critical data.
This can be particularly useful in environments where concurrent access to data is a concern.

(iv) Hashability: Tuples are hashable if all their elements are hashable. This means tuples can be used as keys in dictionaries and as elements of sets, while lists cannot.

ex: person = ('Ankit', 22)
my_dict = {person: 'value'} # valid use of tuple as a key.

(v) Fixed Structure: Tuples are often used to represent fixed collections of items with a specific structure, like coordinates, RGB colors, or database records. in these cases, the structure of the data is known and doesn't need to change.

ex: color = (255, 0, 0) # Red

These are just some advantages of tuples. The best choice between a tuple and a lists depends on the specific requirements of the program.

Q2. Define a function named 'calculate_average' that takes a list of numbers as input and return the avg.

Ans
```
def calculate_average(numbers):
    if not numbers:
        return 0

    total = sum(numbers)
    return total / len(numbers)
```

Apr 9, 2024, 22:36

```
my-numbers = [10,20,30,40,50]
average = calculate_average(my-numbers)
print("Average:", average)
```

O/P: Average : 30

Q3. (i) Implement a recursive function to calculate the nth fibonacci number. ~~Create a~~

(ii) create a python fun called 'merge_dicts' that merges two dictionaries and returns the result.

Ans

(i)

```
def fibonacci(n):
    if n<=1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2)

n= int(input("Enter the value of n to find nth fibonacci number:"))

if n<0:

    print("Invalid input! Please Enter a non-negative Integer.")

else:

    print("The", n, "th fibonacci number is:", fibonacci(n))
```

OUTPUT: Enter the value of n to find the nth fibonacci number: 10

The 10th fibonacci number is: 55

(ii) Ans

```
def merge_dicts(dict1, dict2):
    merged_dict = dict1.copy()

    merged_dict.update(dict2)

    return merged_dict

dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
result = merge_dict(dict1, dict2)
print("Merged dictionary:", result)
```

Output:

Merged dictionary : {'a': 1, 'b': 3, 'c': 4}

(22BCON1068)